

PERFORMANCE OPTIMISATION

Mark Bull

m.bull@epcc.ed.ac.uk



THE UNIVERSITY
of EDINBURGH



Overview

- Optimisation Process
- Profilers
- Optimisation for CPUs
- Optimisation for GPUs



Optimisation process



Why?

- Large computer simulations are becoming common in many scientific disciplines.
- These often take a significant amount of time to run.
 - Sometimes they take *too long*.
- There are three things that can be done
 - Change the science (compromise the research)
 - Change the computer (spend more money)
 - Change the program (this is performance optimisation)

What?

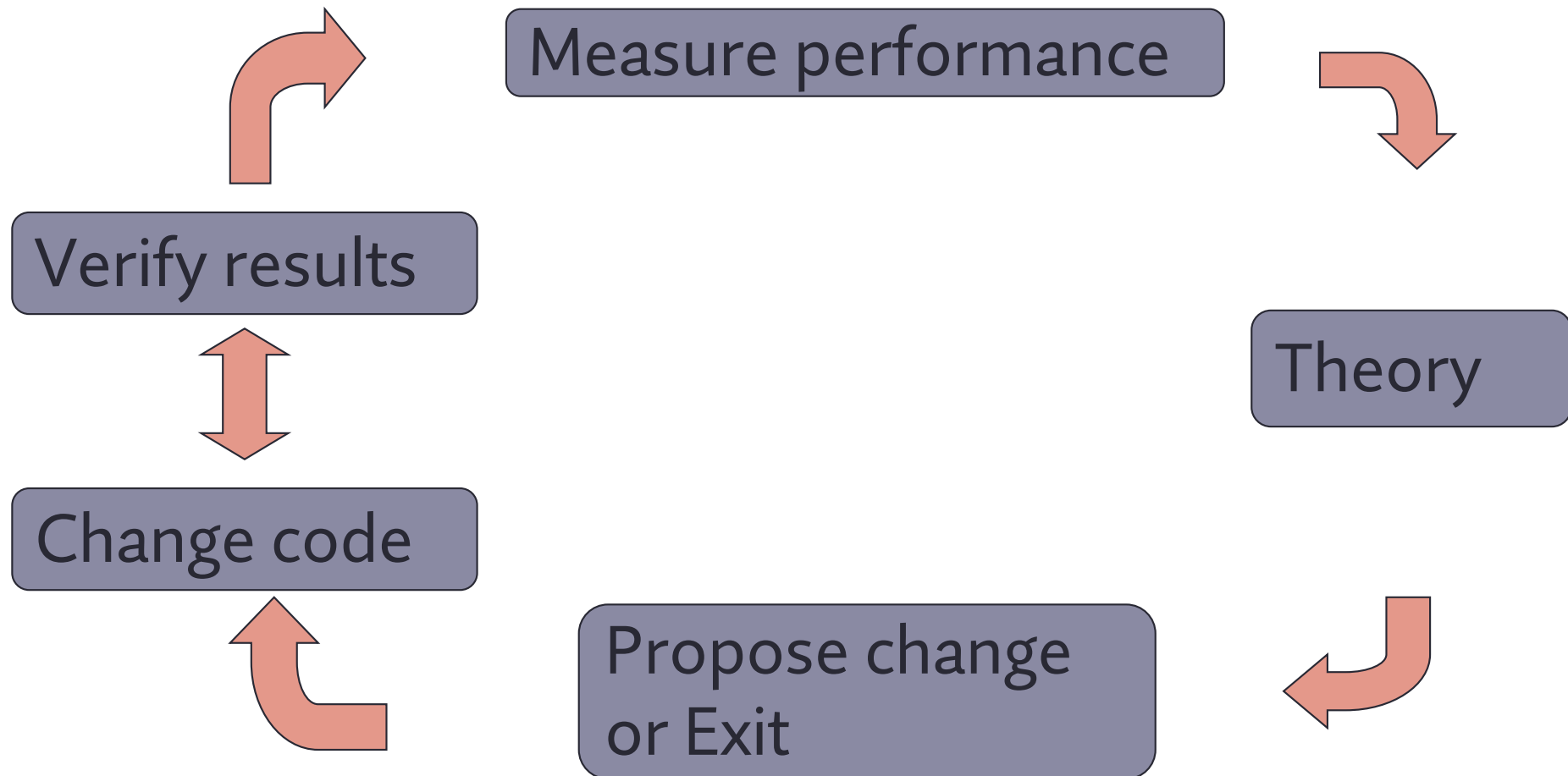
- There are usually many different ways you can write a program and still obtain the correct results.
- Some run faster than others.
 - Interactions with the computer hardware.
 - Interactions with other software.
- **Performance optimisation is the process of making an existing working computer program run faster in a particular Hardware and Software environment.**
 - Converting a sequential program to run in parallel is an example of optimisation under this definition!
 - You may want to optimise for a different quantity like memory or energy use.

When?

- Performance optimisation can take large amounts of development time.
- Some optimisations improve program speed at the cost of making the program harder to understand (increasing the cost of future changes)
- Some optimisations improve program speed at the cost of making the program more specialised and less general purpose.
- It is always important to evaluate the relative costs and benefits when optimising a program
 - This requires the ability to estimate potential gains in advance

How?

- Performance optimisation usually follows a cycle:



Measuring performance

- It is not enough to just measure overall speed
 - You need to know where the time is going.
- There are tools to help you do this
 - They are called profiling tools.
- They give information about:
 - Which sections of code are taking the time
 - Sometimes line by line but usually only subroutines.
 - Sometimes the type of operation
 - memory access
 - floating point calculations
 - file access
- Make sure you understand how variable your results are
 - Are the results down to my changes or just random variation?



Input dependence

- Many codes perform differently with different input data.
- Use multiple sets of input data when measuring performance.
- Make sure these are representative of the problems where you want the code to run quickly.
 - many applications repeat the same calculation over and over
 - can reduce the number of timesteps/iterations for profiling/tuning experiments
 - but make sure you exclude initialisation/finalisation.

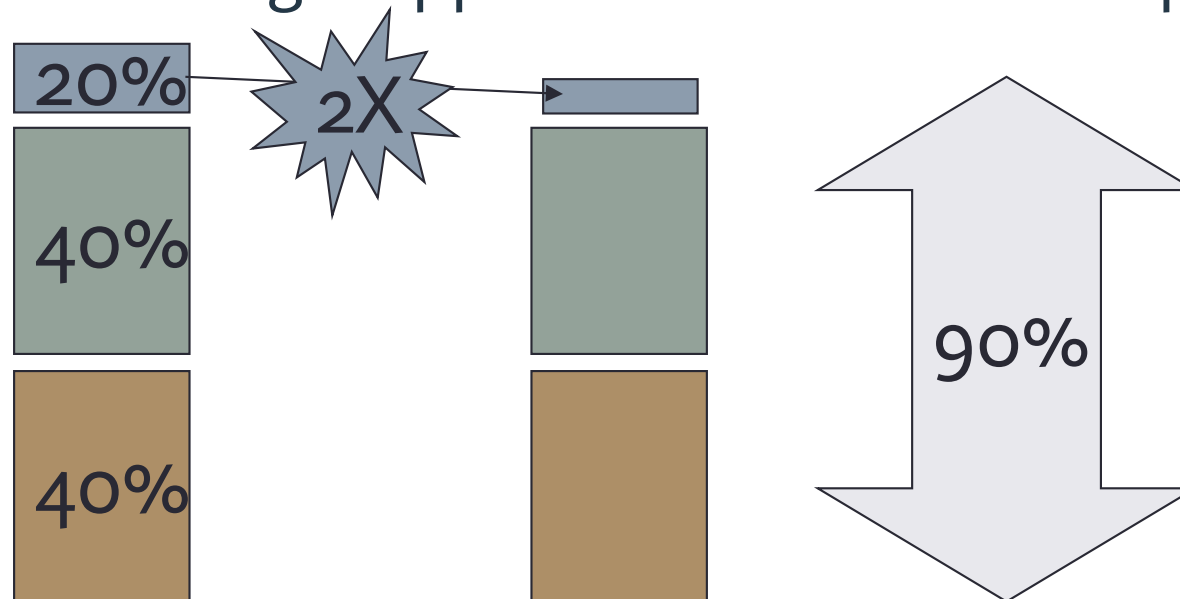
Variability

- If codes that make significant uses of shared resources (like the file-systems) the performance may depend on external factors, such as what other users are doing.



Only optimise important sections

- It's only worth working on parts of the code that take a lot of time.
- Large speed-up of unimportant sections have little impact on the overall picture.
 - Amdahl's law is this concept applied to parallel processing.
 - Same insight applies to other forms of optimisation.



Theory

- Optimisation is an experimental process.
- You propose reasons why a code section is slow.
- Make corresponding changes.
- The results may surprise you
 - Need to revise the theory
- Never “optimise” without measuring the impact.

Exit?

- It is important to know when to stop.
- Each time you propose a code change consider:
 - The likely improvement
 - Code profile and Amdahl's law helps here.
 - Take account of how much use you expect for the optimised code. Single use programs are rarely worth optimising.
 - The likely cost
 - Programming/debugging time.
 - Delay to starting simulation
 - “Damage” to the program

Changing code

- Many proposed changes will turn out not be useful.
- You may have to undo your changes.
 - At the very least keep old versions
 - Better to use revision control software.
- Always check the results are still correct !!
 - No point measuring performance if the results are wrong
 - A good test framework will help a lot

Damaging code

- Performance changes can damage other desirable aspects of the code.
 - Loss of encapsulation.
 - Loss of clarity
 - Loss of flexibility
- Think about down-side of changes.
- Look for alternative changes with same performance benefit but less damage.

Key points

- Optimisation tunes a code for a particular environment
 - Not all optimisations are portable.
- Optimisation is an experimental process.
- Need to think about cost/benefit of any change.
- Always verify the results are correct.

Experimental frameworks

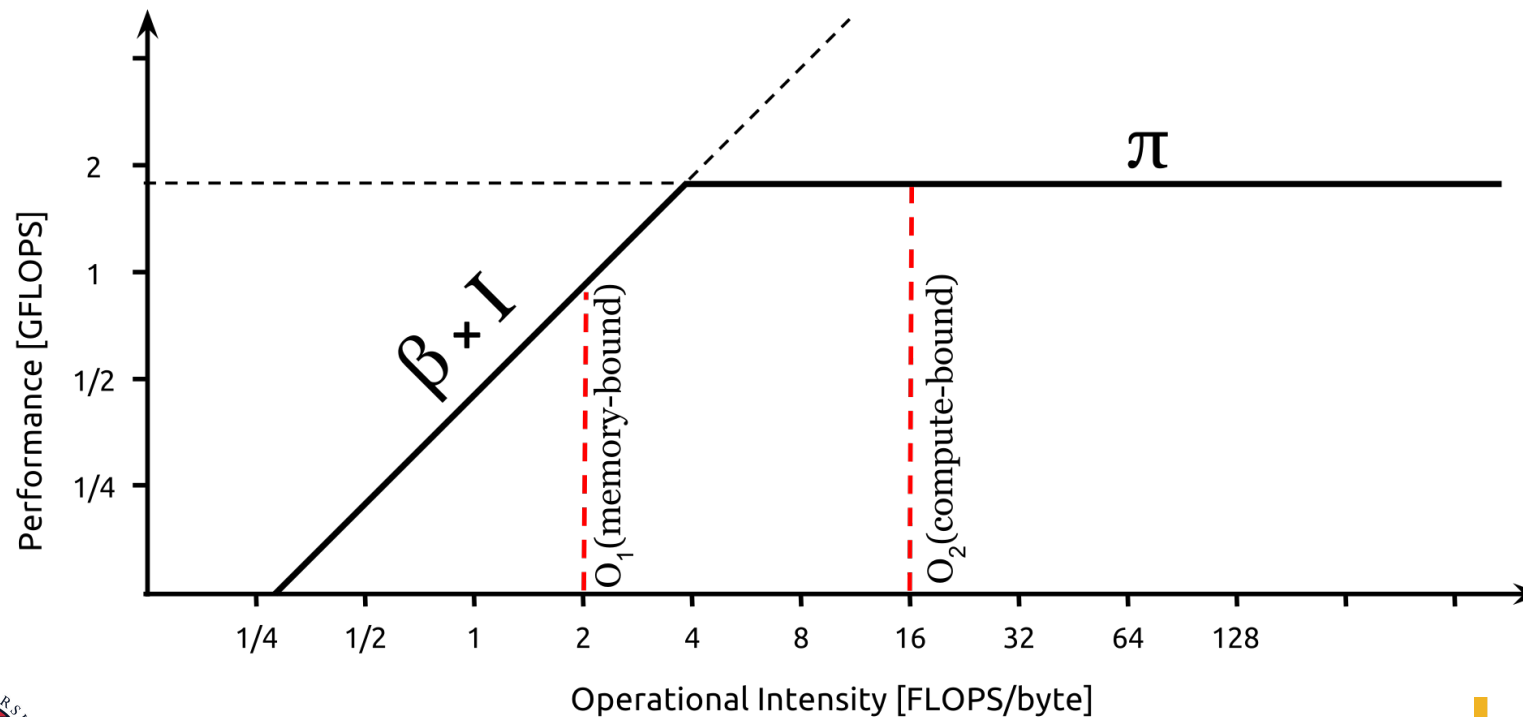
- Like any experiment you need to keep good records.
- You will be generating large numbers of different versions of the code.
 - You need to know exactly what the different version were.
 - How you compiled them.
 - Did they get the correct answer.
 - How did they perform.
- You may need to be able to re-run or reproduce your experiments
 - You discover a bug
 - A new compiler is released
 - A new hardware environment becomes available.
 - Etc.

Making things easier

- Keep everything under version control (including results)
- Script your tests so they are easy to run and give a clear yes/no answer.
- Write timing data into separate log-files in easily machine readable format.
- Keep notes.

Knowing when to stop

- Hardware metrics
- Estimating program requirements
- Roofline model



Compiler Optimisation

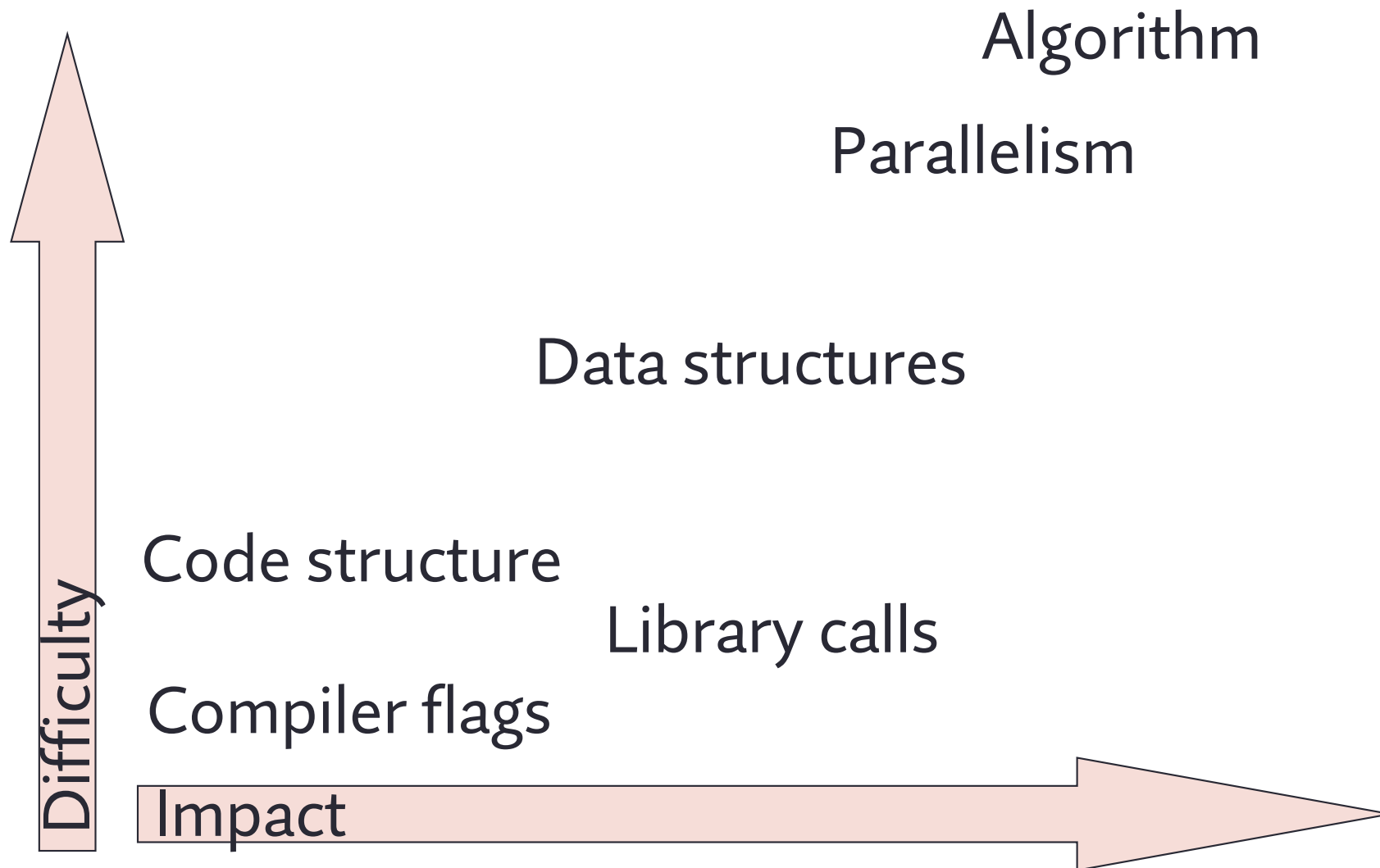
- Automatic optimisation performed by the compiler.
- Compiler takes the source code you give it and tries to find best machine code implementation for the target hardware.
 - Compiler only has limited information about the target hardware
 - Constrained by the source code you give it. Can only make changes allowed within the language specification.
 - Compiler is not intelligent but is much better at some kinds of optimisation than a human programmer.
 - Compiler can afford to optimise the entire program.



Hand optimisation

- Changing the source code to improve performance.
- Used to augment compiler/auto-tuning not as a substitute.
 - Compiler and auto-tuning code is part of the environment you optimise for.

Types of optimisation



Compiler flags

- Easiest thing to change are the compiler flags
- Most compilers will have good optimisation by default.
 - Some compiler optimisations are not always beneficial and need to be requested explicitly.
 - Some need additional guidance from user (e.g. inter-file in-lining)
 - Some break language standards and need to be requested explicitly
 - E.g. $a/2 \rightarrow a*0.5$ is contrary to Fortran standard but is usually safe.
 - Usually worthwhile to read compiler manual pages before optimising.

Library calls

- The easiest way to make a big impact on code performance is to re-use existing optimised code.
- Libraries represent large amount of development effort
 - Somebody else has put in the effort so you don't have to,
- Code libraries exist for commonly used functionality (e.g. linear algebra, FFTs etc.).
 - Often possible to access highly optimised versions of these libraries.
 - Even if the application code does not use a standard library it is often easy to re-write to use the standard calls.

Using libraries

- Don't just use libraries blindly.
- Read the documentation.
 - Learn what the library is capable of.
 - Libraries often have multiple ways of doing the same thing. You need to understand which is best for your code.
- Many modern libraries support auto-tuning.
 - Can run internal experiments to automatically optimise the implementation for a given problem.
 - Application programmer needs to cooperate on this.

Algorithm

- The biggest performance increases typically require a change to the underlying algorithm.
 - Consider changing an $O(N^2)$ sort algorithm to a $O(N \log(N))$ algorithm.
 - This is a lot of work as the relevant code section usually needs a complete re-write.
- A warning
 - The complexity of an algorithm $O(N)$, $O(\log(N))$, $O(N \log(N))$ etc. is related to number of operations and is not always a reliable indication of performance.
 - Pre-factor may make a “worse” algorithm perform better for the value of N of interest.
 - The “worse” algorithms may have much better cache re-use or exhibit more available parallelism.

Data structures

- Changing the programs data structures can often give good performance improvements
 - These are often global changes to the program and therefore expensive.
 - Code re-writing tools can help with this.
 - Easier if data structures are reasonably opaque, declared once
 - objects, structs, Fgo types, included common blocks.
 - As memory access is often the major performance bottleneck the benefits can be great.
 - Improve cache/register utilisation.
 - Avoid pointer chasing
 - May be able to avoid memory access problems by changing code structure in key areas instead.

Code structure

- Most optimisations involve changes to code structure
- Often overlap with optimisations attempted by the compiler.
 - Usually better to help the compiler to do this than perform change by hand.
- Easier to implement than data changes as more localised.
 - Performance impact is often also smaller unless the code fragment is a major time consumer.
- Performance improvement often at the expense of code maintainability.
 - Try to keep the unoptimised version up to date as well.

Parallelism

- Parallel computing is usually an optimisation technique
 - Multiple processors are used to improve run-time
- Large numbers of processors can result in very significant performance improvements.
 - Software effort to develop parallel code can be very high.
 - Code runs faster in real time but may consume more CPU cycles in total.

Instruction level parallelism

- Most modern CPUs include some kind of parallel instructions
 - SIMD instructions where one instruction performs multiple operations.
 - pipelined or superscalar architectures where multiple (independent) instructions are executing at the same time.
- Largely the compiler finds and exploits this kind of parallelism automatically
 - ...therefore generally easy to exploit
 - may still have to change the source code to:
 - increase the parallelism available to the compiler
 - make it easier for the compiler to recognise the parallelism

Shared memory parallelism

- Multiple processors share a single memory system.
 - Data structures can be shared and need not be changed.
 - Only the work needs to be distributed (typically using threads).
- Typically the most flexible form of multi-processor parallelism
 - Parallelism can be introduced incrementally, only critical routines need to be parallel.
 - Often can add parallel directives to a serial code, automatically keeping parallel and serial versions in sync.
- Contention for hardware resources (memory bandwidth, cache space) can limit performance

Distributed memory parallelism

- Processors have private memory systems and perform explicit communications
 - Message passing, remote memory access calls etc.
 - Usually large software development task.
 - The whole code has to run in parallel.
 - Very little if any support from the compiler.
 - Data structures usually need changing. Exceptions:
 - Regular domain decomposition: serial code data structures often ok, just have to be smaller. Boundary condition code needs to be changed.
 - Replicated data: Each node keeps a private copy of the global data structures. Easy to implement but often does not scale well.
- Distributed memory codes can easily run on shared memory systems, but not vice-versa

Accelerators

- Special purpose hardware
 - Very good at some types of operations
 - Can't run the entire code.
 - GPUs commonest example
- Main CPU and accelerator handle different sections of the code.
 - Usually some overhead in switching control
 - Often data has to be copied between devices
 - Different memory layouts might be appropriate for each device
- Trend towards greater compiler support for acceleration
 - Programmer still needs to understand both architectures to get best performance.

Summary

- There are many ways to optimise a program
 - need to select the most appropriate for the situation
 - doing nothing is an option
- Must always weigh the benefit (faster execution) against costs:
 - programmer effort
 - loss of maintainability
 - loss of portability
- Performance measurement, analysis and testing are key parts of the process.



Profiling



What is profiling?

- Analysing your code to find out the proportion of execution time spent in different routines.
- Essential to know this if we are going to target optimisation.
- No point optimising routines that don't significantly contribute to the overall execution time.
 - can just make your code less readable/maintainable

Code profiling

- Code profiling is the first step for anyone interested in performance optimisation
- Profiling works by instrumenting code at compile time
 - thus it's (usually) controlled by compiler flags
 - can reduce performance
- Standard profiles return data on:
 - Number of routine calls
 - Amount of time spent in sections of code
 - Exclusive time – time spent in this routine only
 - Inclusive time – time spent in this routine and all routines called from here

Sampling and tracing

- Many profilers work by sampling the program counter at regular intervals (normally 100 times per second).
 - low overhead, little effect on execution time
- Builds a statistical picture of which routines the code is spending time in.
 - if the run time is too small ($< \sim 10$ seconds) there aren't enough samples for good statistics
- Tracing can get more detailed information by recording some data (e.g. time stamp) at entry/exit to functions
 - higher overhead, more effect on runtime
 - unrestrained use can result in huge output files and overwhelm analysis tools

The Golden Rules of profiling

- Profile your code

The compiler/runtime will **NOT** do all the optimisation for you

- Profile your code yourself

- Don't believe what anyone tells you. They're probably wrong.

- Profile on the hardware you want to run on

- Don't profile on your laptop if you plan to run on a supercomputer.

- Profile your code running the full-sized problem

- The profile will almost certainly be qualitatively different for a small test case.

- Keep profiling your code as you optimise

- Concentrate your efforts on the thing that slows your code down.
- This will change as you optimise.
- So keep on profiling...

Limitations of profilers

- Statistical variation from sampling
- Profiling can perturb execution times
- Routine name mangling
- Profilers have to make choices about where in the call tree to assign time
 - complicated by function inlining, in-built libraries
- Time spent in external libraries may not be captured correctly
 - may need to re-build the library with profiling enabled
- Output can be confusing for multi-threaded programs
 - can time is accumulated across all threads
- Can be hard to compare two (or more) profiles
- Large/complex application codes can break profiling tools

Profiling example – CrayPAT with GNU compiler

Table 1: Profile by Function

Samp%	Samp	Imb. Samp	Imb. Samp%	Group Function PE=HIDE Thread=HIDE
100.0%	20,011.2	--	--	Total
52.9%	10,590.2	--	--	USER
5.5%	1,091.8	335.2	23.6%	__matrix_vector_kernel_mod_MOD_matrix_vector_code_r_double
4.4%	887.9	104.1	10.5%	__apply_mixed_lu_operator_kernel_mod_MOD_apply_mixed_lu_operator_code
2.8%	563.0	48.0	7.9%	__pressure_gradient_bd_kernel_mod_MOD_pressure_gradient_bd_code
2.4%	481.3	77.7	13.9%	__apply_elim_mixed_lp_operator_kernel_mod_MOD_apply_elim_mixed_lp_operator_code
2.2%	441.2	59.8	12.0%	__opt_apply_variable_hx_kernel_mod_MOD_opt_apply_variable_hx_code_r_single
1.8%	356.2	72.8	17.0%	__scaled_matrix_vector_kernel_mod_MOD_scaled_matrix_vector_code
1.7%	342.6	37.4	9.9%	__hydrostatic_kernel_mod_MOD_hydrostatic_code
1.4%	287.3	41.7	12.7%	__tri_solve_kernel_mod_MOD_tri_solve_code_r_single
1.4%	275.6	46.4	14.4%	__dg_inc_matrix_vector_kernel_mod_MOD_dg_inc_matrix_vector_code_r_single
1.3%	252.7	140.3	35.8%	__poly1d_vert_w3_reconstruction_kernel_mod_MOD_poly1d_vert_w3_reconstruction_code
1.2%	242.5	65.5	21.3%	__apply_helmholtz_operator_kernel_mod_MOD_apply_helmholtz_operator_code
1.1%	222.9	50.1	18.4%	__r_solver_field_vector_mod_psy_MOD_invoke_0._omp_fn.0
1.0%	203.2	63.8	23.9%	__psykal_lite_mod_MOD_invoke_copy_to_rtran._omp_fn.0
29.9%	5,987.8	--	--	ETC
14.3%	2,852.9	360.1	11.2%	__memcpy_avx_unaligned_erms
3.3%	655.5	450.5	40.8%	gomp_team_barrier_wait_end
2.6%	529.1	215.9	29.1%	cxip_cq_progress
1.6%	330.0	190.0	36.6%	cxip_ep_ctrl_eq_progress
1.0%	202.1	83.9	29.4%	_int_free
12.6%	2,517.7	--	--	MPI
4.4%	870.7	182.3	17.4%	MPIDI_Cray_shared_mem_coll_bcast
3.0%	607.3	125.7	17.2%	MPIDI_SHMI_progress
2.4%	479.6	163.4	25.5%	MPIR_Waitall_impl
3.1%	611.5	--	--	FABRIC
2.3%	455.1	189.9	29.5%	ofi_cq_readfrom

Timer calls

- It can also be useful to insert timer calls into the code yourself
 - can time sections inside routines and other sections of interest
 - low overhead, robust and reliable!
 - can make it easy to post-process output
 - sanity check on profiler output
- Make sure you use a timer that measures wall-clock time
 - not CPU time or something else....
- Sum up all the time spent in a block of code
 - plenty of existing libraries to do this



Optimising for CPUs



Processors

- Basic functionality
 - execute instructions to perform arithmetic operations (integer and floating point)
 - load data from memory and store data to memory
 - decide which instructions to execute next
- Arithmetic is performed on values in registers (local storage in the processor)
 - moving data between memory and registers must be done explicitly by load and store instructions
 - separate integer and floating point registers
 - typical size ~100 values
- Basic characteristics:
 - Clock speed
 - Peak floating point capability



Processors (cont.)

- Clock speed determines rate at which instructions are executed
 - modern chips are around 2-3 GHz
 - integer and floating point calculations can be done in parallel
 - can also have multiple issue, e.g. simultaneous add and multiply
 - peak flop rate is just clock rate x no. of floating point operations per clock cycle
- Whole series of hardware innovations
 - pipelining
 - out-of-order execution, speculative computation
 - ...

Moore's Law

- “CPU power doubles every 18 months”
 - strictly speaking, applies to transistor density
- Held true for ~40 years
- People have predicted its demise many times
 - may finally be actually happening
- Increases in power are due to increases in parallelism
 - fine grain parallelism (pipelining)
 - medium grain parallelism (superscalar, SIMD, hardware multithreading)
 - coarse grain parallelism (multiple processors on a chip)
- First two seem to be (almost) exhausted: main trend is now towards multicore

Pipelining

- Key implementation technique for making fast CPUs.
- Execution of instructions is broken down into **stages**.
- Each stage can be executed in one CPU clock cycle
 - all parts of CPU operate at a fixed frequency
- Once a stage has completed for one instruction, it can be executed for the next instruction on the subsequent clock cycle.
- Allows one instruction to be completed per clock cycle, even though the instruction itself may take many cycles to complete.
 - AMD Epyc Rome has 19 stage pipeline
- But not all instructions are pipelined
 - e.g. FP square root, FP divide

Keeping pipelines full

- CPUs try to guess which way code branches will go (branch prediction)
 - keep issuing instructions on the predicted branch and only back out if the prediction was wrong
 - if tests that depend on data values can be hard to predict
 - try to avoid having tests inside innermost loops
- FP divides and square roots are expensive
 - compiler optimisation can help by not repeating the same calculation

SIMD instructions

- Instructions which encode operations on multiple data items
(**S**ingle Instruction **M**ultiple **D**ata)
- Also known as vector instructions

Example:

A simple floating point instruction might encode

$$\mathbf{c} = \mathbf{a} + \mathbf{b}$$

A SIMD floating point instruction could encode

$$\mathbf{c} = \mathbf{a} + \mathbf{b} \text{ and } \mathbf{f} = \mathbf{d} + \mathbf{e}$$

in one instruction.

SIMD

- Most modern processors include support for SIMD instructions
 - e.g. SSE, AVX, AVX2, AVX512 in x86, AltiVec in POWER, Neon, SVE in Arm
- Requires a (substantial) extension to the instruction set
 - load/stores, compares, as well as integer and FP arithmetic
- Requires multiple arithmetic units and additional vector registers to take advantage of them
- The **SIMD width** is the number of operations encoded in a SIMD instruction
 - typically 2, 4 or 8

Exploiting SIMD instructions

- Restrictions on what SIMD memory operations (loads/stores) can do
 - e.g. a 2-word SIMD load of double precision values may require that the values are consecutive in memory and are 16 byte aligned.
- Compiler is responsible for identifying operations which can be combined into SIMD instructions.
- In practice, this is a tough job for the compiler.
 - alignment of data is often not determinable at compile time
 - compiler can generate multiple versions of code and pick the right one depending on the alignment encountered at run time.....
 - may require hints, or even hand-coded assembly

When does the compiler vectorize

- What can be vectorized
 - Only loops
- Usually only one loop is vectorisable in loopnest
 - And most compilers only consider inner loop
- Optimising compilers will use vector instructions
 - Relies on code being vectorisable
 - Or in a form that the compiler can convert to be vectorisable
 - Some compilers are better at this than others
- Check the compiler output listing and/or assembler listing
 - Look for packed AVX/AVX2/AVX512 instructions
 i.e. Instructions using registers `zmm0–zmm31` (512-bit) `ymm0–ymm31` (256-bit) `xmm0–xmm31` (128-bit)
 Instructions like `vaddps`, `vmulps`, etc...

Requirements for vectorisation

- Loops must have determinable (at run time) trip count
 - rules out most while loops
- Loops must not contain function/subroutine calls
 - unless the call can be inlined by the compiler
 - maths library functions usually OK (but not with GNU compilers)
- Loops must not contain branches or jumps
 - guarded assignments may be OK
 - e.g. `if (a[i] != 0.0) b[i] = c * a[i];`
- Loop trip counts needs to be long, or else a multiple of the vector length

- Loops must not have dependencies between iterations
 - reductions usually OK, e.g. `sum += a[i];`
 - avoid induction variables e.g. `idx += 3;`
 - may need to tell the compiler if it can't work it out for itself
- Aligned data is best
 - e.g. AVX vector loads/stores operate most effectively on 32-bytes aligned address
 - need to either let the compiler align the data....
 - ..or tell it what the alignment is
- Unit stride through memory is best

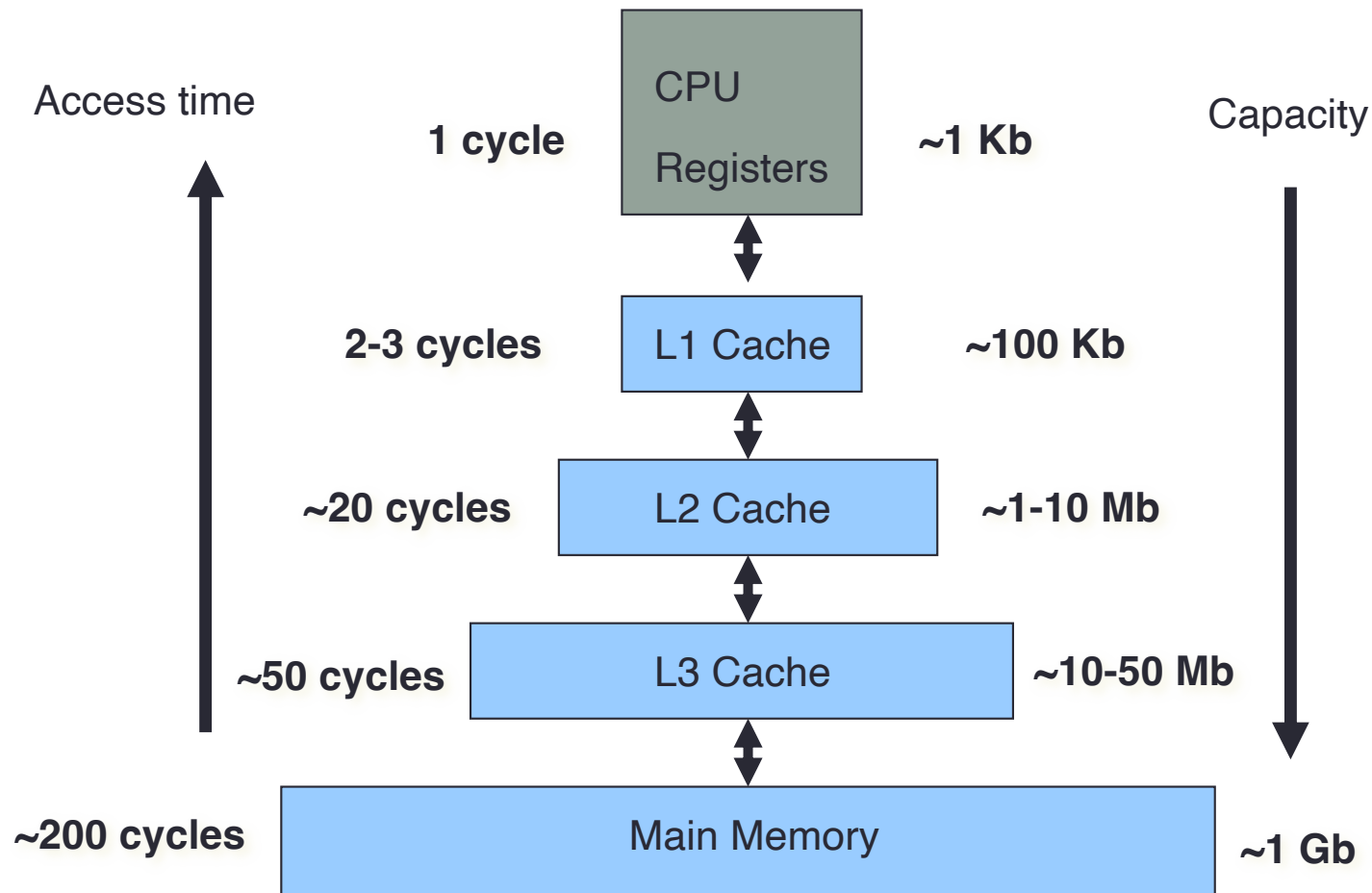
Memory

- Memory speed is often the limiting factor for HPC applications
 - keeping the CPU fed with data is the key to performance
 - most HPC applications attain less than 10% of the CPUs peak floating point performance
- Memory is a substantial contributor to the cost of systems
 - typical HPC systems have a few Gbytes of memory per processor
 - technically possible to have much more than this, but it is too expensive and power-hungry
- Key metrics
 - latency: how long you have to wait for data to arrive
 - bandwidth: how fast it actually comes in
 - ballpark figures: 100's of nanoseconds and a few Gbytes/s

Cache memory

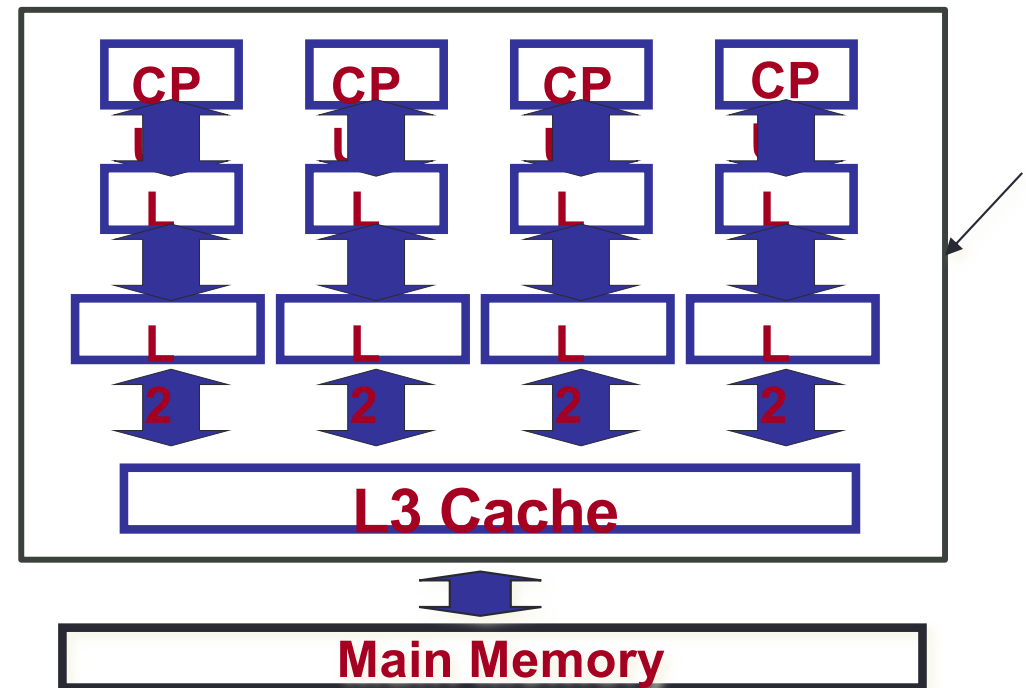
- Memory latencies are very long
 - Typically 100-200 processor cycles
 - fetching data from main memory is 2 orders of magnitude slower than doing arithmetic
- Solution: introduce cache memory
 - much faster than main memory
 - ...but much smaller than main memory
 - keeps copies of recently used data
- Modern systems use a hierarchy three levels of cache

Memory Hierarchy



Multicore chips

- Now commonplace to have lots of cores on a chip.
- Typically, each core has its own Level 1 and Level 2 caches, but the Level 3 cache is shared between some subset of cores



Caches

- Sophisticated book-keeping required
 - need to know where most up-to-date copy is kept
 - flush old data downwards to make space for new data
 - done automatically, transparent to the CPU
 - programs still see simple register/memory model
- Caches only help performance if the application re-uses recently accessed data
 - mostly the programmer's responsibility to order computations so as to maximise the re-use
- Many modern memory systems also do prefetching
 - make (quite simple) guesses as to which data will be used next
 - load these data into caches before the processor requests them

Principal of locality

- Almost every program exhibits some degree of locality.
 - Tend to reuse recently accessed data.

- Two types of data locality:

1. Temporal locality

A recently accessed item is likely to be reused in the near future.

e.g. if x is read now, it is likely to be read again, or written, soon.

2. Spatial locality

Items with nearby addresses tend to be accessed close together in time.

e.g. if $y[i]$ is read now, $y[i+1]$ is likely to be read soon.

How do caches help?

- Cache hold copies of data from main memory locations.
- Cache can hold recently accessed data items for fast re-access.
- Fetching an item from cache is much quicker than fetching from main memory.
 - 1 nanosecond instead of 100.
- For cost and speed reasons, cache is much smaller than main memory.
- Data is moved between caches and memory in units of cache blocks (also called cache lines)
 - 64 bytes is a typical size for cache blocks

What does the processor see?

- All the machinery in the cache is essentially invisible to the processor.
- Assembly code is scheduled assuming the data is in Level 1 cache.
- If it is not (i.e. a **cache miss**), the processor has to wait until the data is found and loaded.
- Processor can continue executing instructions
 - but there is a limit on the number of outstanding memory references.
 - after this, the processor will stop and wait

Arrays

- Many codes loop over array elements
 - Data access pattern is regular and easy to predict
- Good spatial locality achieved by accessing **neighbouring elements** on **consecutive iterations** of the innermost loop.
- Unless loop nest order and array index order match the access pattern may not be optimal for cache re-use.
 - Compiler can potentially address these problems by transforming the loops.
 - But often can do a better job when provided with a more cache-friendly index order.

Bad spatial locality

```
for (j=0; j<M; j++) {
    for (i=0; i<N; i++) {
        a[i][j] += b[i][j];
    }
}
```

```
do i=1,n
    do j=1,m
        a(i,j)=a(i,j)+b(i,j)
    end do
end do
```

Good spatial locality

```
for (i=0; i<N; i++) {
    for (j=0; j<M; j++) {
        a[i][j] += b[i][j];
    }
}
```

```
do j=1,m
    do i=1,m
        a(i,j)=a(i,j)+b(i,j)
    end do
end do
```


Arrays of structs or structs of arrays?

```
struct Part{  
    double x;  
    double y;  
    double z;  
    int index;  
    double mass;  
}  
Part  
data[numParts];
```

Array of structs

```
struct AllParts{  
    double x[numParts];  
    double y[numParts];  
    double z[numParts];  
    int index[numParts];  
    double mass[numParts];  
}  
or  
AllParts data;
```

Struct of arrays

Array of structs:

- May have good temporal locality if there is lots of computation on each struct
- May have poor spatial locality if the most frequent computations don't use most of the fields
- Unfavourable for vector loads/stores
- Natural for OO design

Struct of arrays

- May have better spatial locality (use all data on cache line), but worse temporal locality
- More favourable for vector loads/stores
- Less natural for OO design

High level data structures

- Many modern languages have built in-support for high level data structures such as
 - Lists
 - Trees
 - Sets
 - Maps
 - Etc.
- May be available either as built-in data-types or as standard libraries.
 - Have the same intrinsic advantages/disadvantages as home made equivalents but typically better tested and optimised.

Algorithms and data structures

- Languages and libraries that provide data structures are very helpful
 - but it can be easy to increase the algorithmic complexity of a code by careless/inappropriate use
 - choice of structure may depend on the complexity of the commonest operations you need to do
 - e.g. a sorted list that can be added to
 - adding a item to a vector and resorting it versus inserting into a heap
 - also easy to have unwanted allocations and/or copying

Reducing memory accesses

- Memory accesses are often the most important limiting factor for code performance.
 - Memory bandwidth is a critical resource on modern CPUs
 - especially when all cores are consuming it at the same time
 - Most HPC codes don't achieve more than about 10% of the peak floating point performance of the chip
 - memory accesses are the bottleneck
 - Can also be constrained by memory latency
- Things to look for:
 - Unnecessary temporary arrays.
 - Tables of values that would be cheap to re-calculate

Temporary array example

```
for (i=0; i<M; i++) {  
    a[i] = ....;  
}
```

```
for (i=0; i<M; i++) {  
    b[i] = ....;  
}
```

```
c = dot_product(a, b, M);
```

Bad if arrays do not fit in cache

```
c = 0.0;
```

```
for (i=0; i<M; i++) {  
    a = ....;  
    b = ....;  
    c += a*b;  
}
```

Better, but less intuitive code

- Note: libraries such as C++ STL can make it hard to avoid this!

Cache blocking

- Suppose we need to perform a sequence of calculations on each element of a large data structure

```
for (i=0 ; i<M ; i++) {  
    calc1 (&a[i]) ;  
    calc2 (&a[i]) ;  
    calc3 (&a[i]) ;  
}
```

Reuse good, vectorisation poor

```
calc1all (a, M) ;  
calc2all (a, M) ;  
calc3all (a, M) ;
```

Reuse poor, vectorisation good



Cache blocking

```
for (i=0 ; i<M ; i+=BLOCK) {  
    calc1all (&a[i] , BLOCK) ;  
    calc2all (&a[i] , BLOCK) ;  
    calc3all (&a[i] , BLOCK) ;  
}
```

- Choose **BLOCK** such that one section of **a** fits in cache
- Good reuse and vectorisation!

Summary

- Structure the code so that the compiler can find instruction-level parallelism in loops
 - and therefore use vector instructions and do other optimisations
- Structure the code to make best re-use of data
 - exploit both temporal and spatial locality
- Don't forget the compiler is always there
 - try to help it do its job, rather than do it in the code
- Use libraries where possible
 - but be aware of possible performance pitfalls

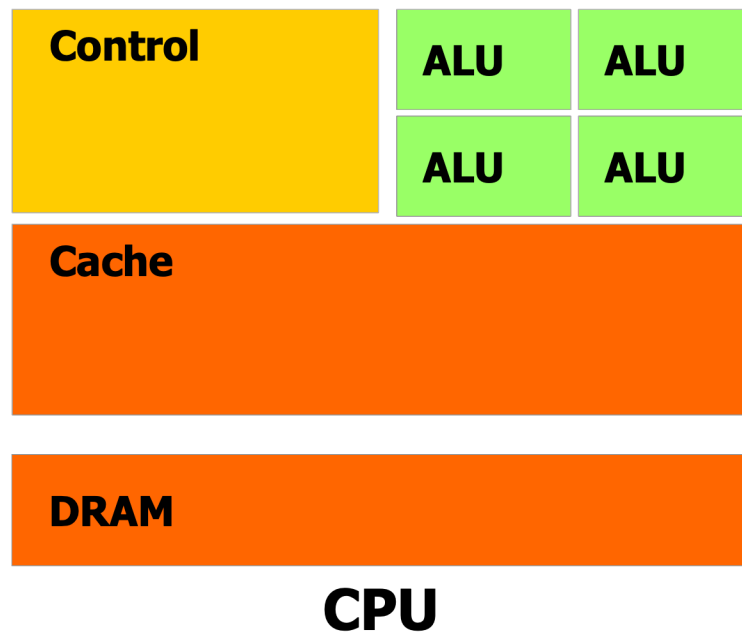


Optimising for GPUs



CPU vs GPU Hardware

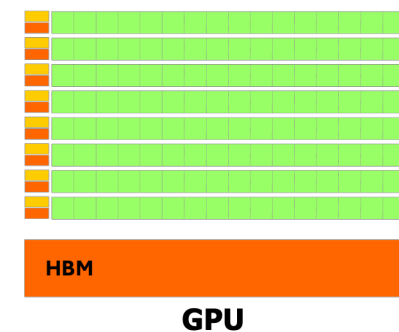
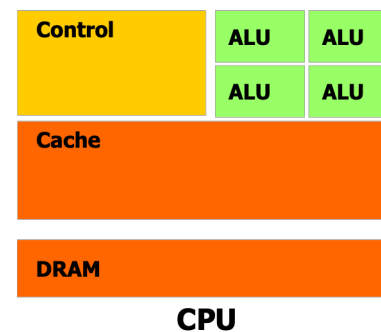
- Much more area dedicated to compute



Modified from NVIDIA, CC BY 3.0 <<https://creativecommons.org/licenses/by/3.0/>>, via Wikimedia Commons

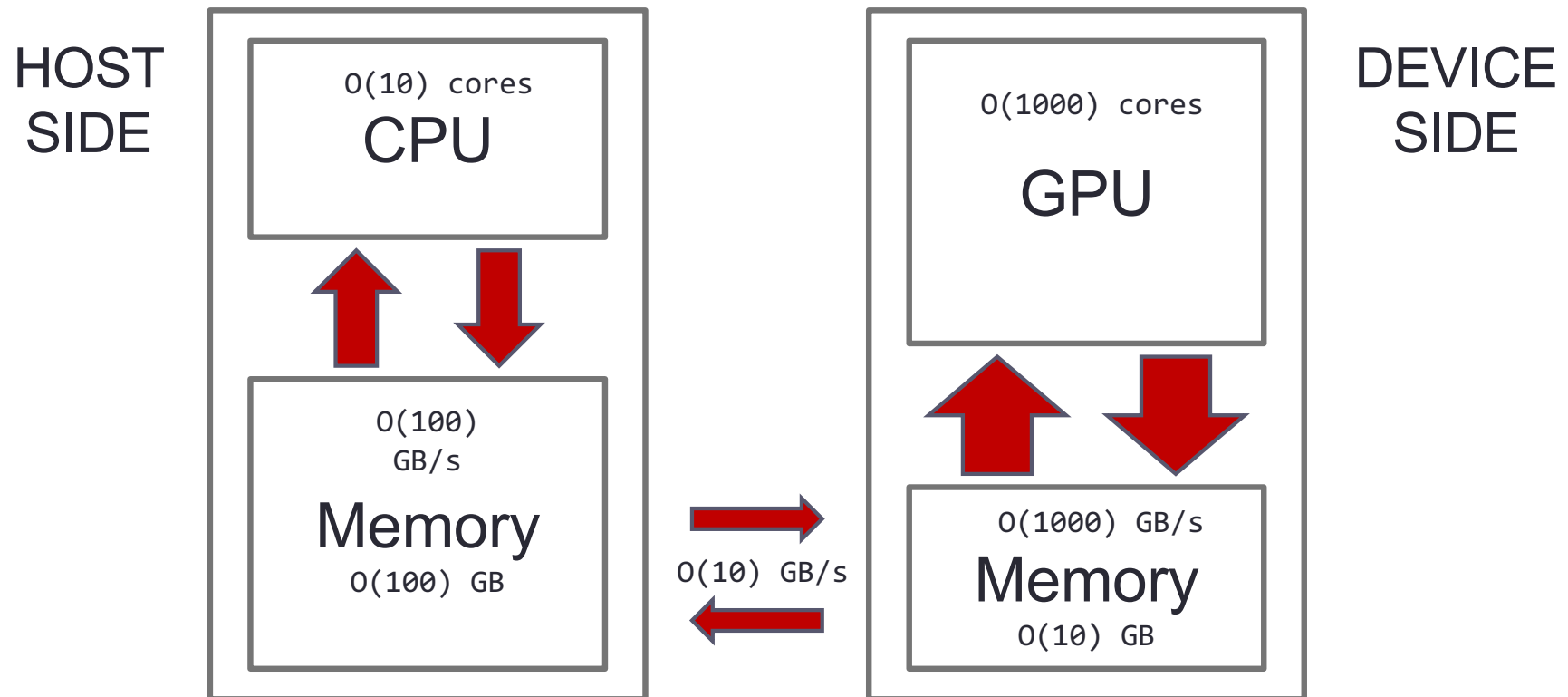
CPU vs GPU Hardware

- CPU computational units are “bigger” but smaller in number
- CPU cores are faster and “smarter” than a GPU core
 - e.g. out-of-order executions, branch prediction, speculative execution
- GPUs specialise in floating point operations:
 - e.g. multiply-add, fused multiply-add, tensor core, ray tracing core
- Performance of a GPU does not come from the processing capabilities of individual cores, rather the massive parallelism



Modified from NVIDIA, CC BY 3.0 <<https://creativecommons.org/licenses/by/3.0/>>, via Wikimedia Commons

Host Device Picture



Data movement

- Moving data between CPU and GPU is slow.
 - Unless the GPU has hardware support for unified memory (e.g. NVIDIA GraceHopper)
- Keep data on the GPU as much as possible
- Avoid un-necessary data transfers
 - only transfer back what is really needed on the CPU side

- GPU cores are grouped into blocks
 - Streaming multiprocessors (SM) – NVIDIA
 - Compute units (CU) – AMD
- O(100) CUs/SMs in a GPU with 32/64 cores each
- GPU devices are **high latency, high bandwidth** devices
- You need to have enough independent tasks to fill all memory/compute units on the devices
- You need to have enough independent tasks to exploit the high bandwidth and hide latency
- Typically need ~4 software threads per core for maximum efficiency
 - Tens of thousands of threads
- For loop nests, this often means exploiting parallelism in more than one loop.

Register pressure

- Each thread needs a certain number of registers to store temporary values
- Finite number of registers per CU/SM
 - fewer registers per core than on CPUs
- This can limit the number of threads that are executed concurrently on a CU/SM
- If more registers are needed than are available, the compiler can decide to spill the registers onto shared or global memory.
 - but register memory is much faster than shared or global memory
- Split big loop bodies into multiple loops with smaller bodies.
 - Smaller loop bodies might need smaller number of registers
- Smaller loops might decrease the amount of parallelism, so might not improve performance.
- May require additional temporary arrays to transmit values between loops
 - increases memory usage, can cause more pressure on cache space and/or memory bandwidth

Single Instruction Multiple Data (SIMD)

- All the GPU cores in a CU/SM execute exactly the same operation, but over different data.
 - c.f. vector instructions on CPUs
- Rather than N clock steps to process N data points, N compute units process N data points in a single clock step (if sufficient cores).
- Threads are grouped in wavefronts / warps
- The number of threads in a wavefront/warp is architecture dependent
 - For AMD GPUs a wavefront is usually 64 threads
 - For NVIDIA GPUs a warp is usually 32 threads
- All threads in a wavefront/warp must always execute the same instruction

Branching

- What if different threads need to execute different instructions ?

```
#pragma omp target teams \  
distribute parallel for  
for (int i=0; i<n; i++){  
    if (i%2 == 0) {  
        x[i] = a;  
    }  
    else {  
        x[i] = b;  
    }  
}
```

C/C++

```
!$omp target teams distribute parallel do  
do i=1,n  
    if (modulo(i,2) == 0) then  
        x(i)= a  
    else  
        x(i)=b  
    endif  
end do  
!$omp end target teams
```

F

Branching

- Set x to a on all threads, but mask odd threads



```
if (i%2 == 0)
{
    x[i] = a
}
```

C/C++

- Set x to b on all threads, but mask even threads



```
else
{
    x[i] = b;
}
```

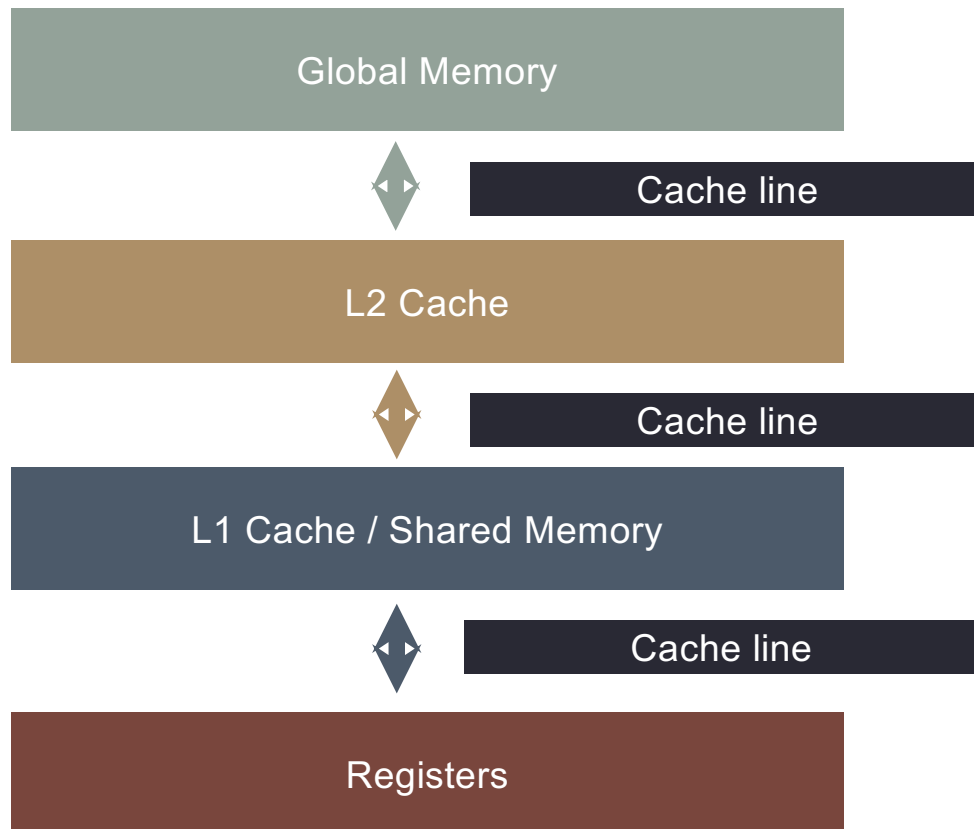
C/C++

- Need to run two instructions per thread instead of one instruction per thread

Branching

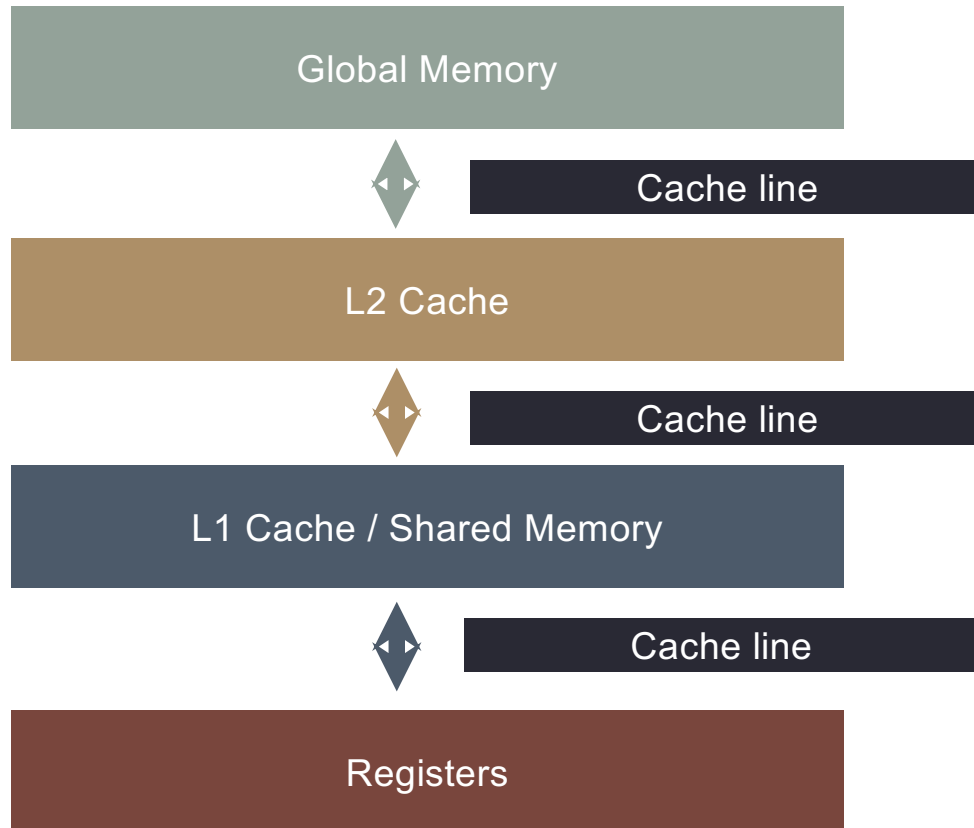
- For each instruction, run the instruction on all threads, but mask the result of threads that do not need to run the instruction
- Each branch executed by all threads: leads to loss of parallelism
- Can strongly limit the performance of some scientific codes.
- An example of codes strongly affected are Monte Carlo codes.
 - Monte Carlo codes often execute a random event per thread. All threads then execute different instructions at the same time, leading to serialization

Memory usage



- Aim to reduce the amount of data traffic between layers
- Try to reuse as much as possible data already in the layers (no different to normal CPUs!)
- GPUs have less cache per core than CPUs
- Less important provided there is enough parallelism to hide main memory latency

Memory usage



- All data is transferred in bundles of a fixed size, called a **cache line**
- Cache line size depends on hardware. Usually 64 bytes or 128 bytes
- If threads in the same wavefront/warp access the same cache line, only one load instruction is needed: accesses are **coalesced**

Coalesced memory access

```
#pragma omp target teams distribute parallel for  
for (int i=0; i<n; i++){  
    a[i]= b[i] + c;  
}
```

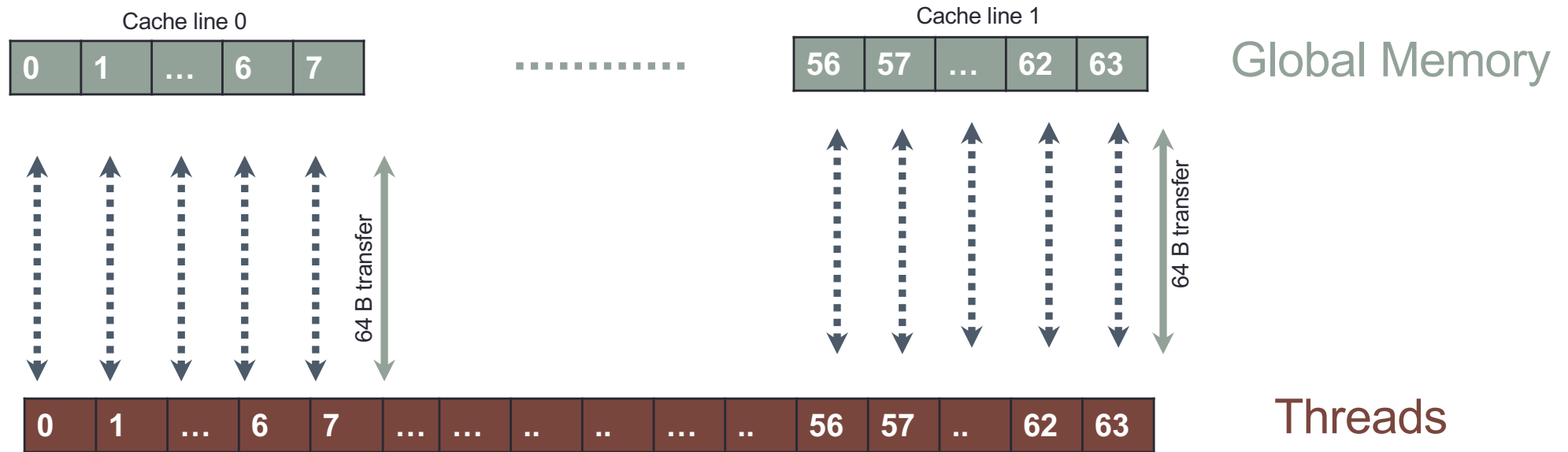
C/ C++

```
!$omp target teams distribute parallel do  
    do i=1,n  
        a(i) = b(i) + c  
    end do  
!$omp end target teams
```

F



Coalesced memory access



- 64 double precisions elements loaded from b
- 8 LOAD instructions per wavefront from b
- $64B \times 8 = 512B$ transferred from b

Coalesced memory access

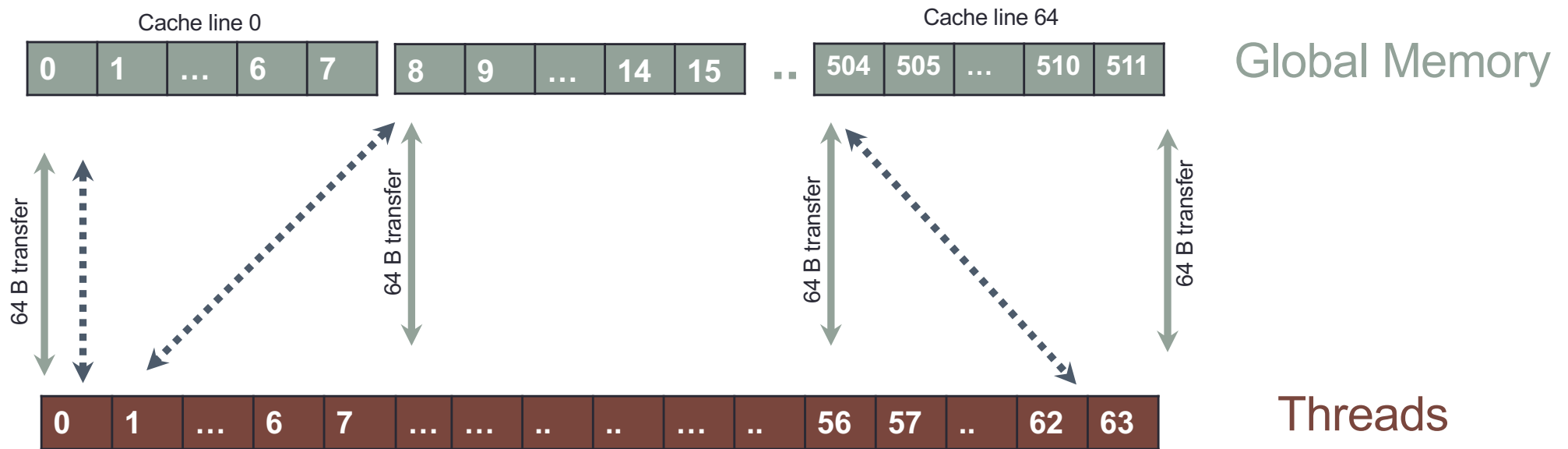
```
#pragma omp target teams distribute parallel for
for (int i=0; i<n; i++)
{
    a[i]= b[i*8] + c;
}
```

C/C++

```
!$omp target teams distribute parallel do
    do i=1,n
        a(i) = b(i*8) + c
    end do
!$omp end target teams
```

F

Coalesced memory access



- 64 double elements loaded from b
- 64 LOAD instructions per wavefront from b
- $64 \text{ B} \times 64 = 3.9 \text{ KiB}$ transferred from b

Coalesced memory access

- Want neighbouring threads/cores to access neighbouring words in memory
- Note that this pattern is bad for CPUs!
 - causes cache invalidations (false sharing)
- May need to use a different layout for data on the GPU that for CPU

Use libraries

- Lots of available libraries for common scientific operations
- Some libraries are vendor independent (will work with AMD, NVIDIA, INTEL ...) e.g. MAGMA
- Some are vendor dependent. But you can switch between implementations at build or run time, based on available hardware (e.g. rocBLAS, cuBLAS ...)

Summary

- Ensure there is enough parallelism to fill all the compute units
- Avoid branches
- Optimize memory accesses, make sure neighbouring threads access neighbouring data elements in memory
- Have kernels sufficiently big to mitigate launch cost, but sufficiently simple not to run out of registers

Reusing this material



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

This means you are free to copy and redistribute the material and adapt and build on the material under the following terms: You must give appropriate credit, provide a link to the license and indicate if changes were made. If you adapt or build on the material you must distribute your work under the same license as the original.

Note that this presentation contains images owned by others. Please seek their permission before reusing these images.